

函数卡 · 遇到再查，别背

建 2026.09.24 卡数 79 张（79 张已实测） 怎么用 搜名字 → 看场景 → 复制代码

这套卡**不是拿来背的**——是写代码卡住时用来查的。每张卡上的代码和输出，都由 `tools/build_fn_cards.py` 真的跑过一遍再写进来，所以卡里不会出现“看着合理、其实报错”的示例。

你在这里 > 阶段 A > ② 编程主线 > 函数卡

全局思维导图 ↗

本周书 ↗

用法 这套卡怎么用（30 秒）

你看到的	意思
本周	现在就得会 —— 第 3 课 + D8 作业里直接要用的。其余的先不用看。
本月	这个月大概率撞到（排序、聚合、文件、JSON）。撞到再回来查。
以后	先知道“有这么个东西”，别现在学。看到名字能想起来就行。
实测输出（绿块）	脚本跑出来的真实结果，不是手写的。红块 = 故意让它报错的现场，因为报错信息本身就是要认的东西。

⚠ 抄之前先改表名。卡里的示例跑在一份样板数据上（选修课三张表 `students / courses / enrollments` + 第 3 课的微信群三张表 `persons / groups / group_members`）——换成你自己的库，表名和列名都要跟着改。这正是第 3 课教的「换皮」。

内置函数**不做成背诵卡**（那条线已经关掉）——只留 8 张最常撞到的当查询用。要补哪些，说一声就加。

SQL 42 张

SELECT ... FROM ... 本周 SQL

所有查询的骨架：先说「要哪几列」，再说「从哪张表」

SELECT name, class FROM students;

实测输出

name	class
韩立	讯飞班
小明	讯飞班
小红	计科1班
小刚	计科1班
小美	计科1班

（5 行）

📌 第3课① · 任何一条 SQL 都长这样

⚠ SELECT * 能跑但别常用 —— 列的顺序以后会变，代码会悄悄错位

WHERE 本周 SQL

先在脑子里选出全部，再一条条筛掉不要的

SELECT name, class FROM students WHERE class = '讯飞班';

实测输出

name	class
韩立	讯飞班
小明	讯飞班

（2 行）

📌 第3课 · 筛选；D8 作业至少要用一次

⚠ 筛完只剩 2 行不是因为「表里只有 2 行」，是 WHERE 把它们筛掉了

ON / WHERE 分工 本周 SQL

ON = 怎么配对（横着拼），WHERE = 要哪些（竖着筛）

```
SELECT s.name, e.cid
FROM students s
JOIN enrollments e ON s.id = e.sid
WHERE e.cid = 1;
```

实测输出

name	cid
韩立	1
小明	1

(2 行)

📌 第3课④·全课最该分清的一对

⚠️ 把 `e.cid = 1` 挪到 ON 里，INNER JOIN 结果一样——但换成 LEFT JOIN 就天差地别（留在 ON 里 = 右表筛完再拼，放 WHERE 里 = 拼完再筛，会把 NULL 行筛掉）

COUNT(*) / COUNT(列) 本周 SQL

COUNT(*) 数行；COUNT(列) 只数**这一列不是 NULL**的行

```
SELECT COUNT(*) AS rows_all, COUNT(grade) AS
rows_graded FROM enrollments;
```

实测输出

rows_all	rows_graded
6	5

(1 行)

📌 第3课⑥ 诊断第一步：单独数绳子表

⚠️ 两者不一样就说明这列有 NULL——这个差值本身就是线索，不是 bug

JOIN ... ON（两表） 本周 SQL

拿一边的 id 去另一边配对，配对成功才出一行

```
SELECT s.name, e.cid
FROM students s
JOIN enrollments e ON s.id = e.sid;
```

实测输出

name	cid
韩立	1
韩立	2
小明	1
小明	3
小红	5
小刚	4

(6 行)

📌 第3课④·本课核心

⚠️ 结果行数 = 配对成功的次数，不是任何一张表的行数

三表 JOIN（中间那张是「绳子」） 本周 SQL

行数由中间那张绳子表说了算；两头都只是「换名字」

```
SELECT s.name AS student, c.name AS course,
e.grade
FROM students s
JOIN enrollments e ON s.id = e.sid
JOIN courses      c ON e.cid = c.id;
```

实测输出

student	course	grade
韩立	数据库	88.5
韩立	高数	76.0
小明	数据库	92.0
小明	机器学习	81.0
小红	英语	59.0
小刚	体育	NULL

(6 行)

📌 第3课④·第6章换皮就是把它改名

⚠️ 三张表的顺序可以换，但**每一句 ON** 必须紧跟着它要拼的那张表

坑：两边都有 `id`，不给表起别名 **本周** **SQL**

两张表都有 `id` 列时，`ON students.id = courses.id` 里的 `id` 到底是谁的？

```
SELECT id, name FROM students JOIN courses ON
students.id = courses.id;
```

实测：真的会这样报错

`OperationalError: ambiguous column name: id`

📌 报错现场 —— 别名就是为了解决它

⚠️ `ambiguous column name` 的正确解法是给表起别名然后写全：`s.id`、`c.id`

坑②： `id` 重叠 → 照样出数据（最危险） **本周** **SQL**

`students.id` 和 `courses.id` 都从 1 开始，接反了照样配得上 —— 于是给你一张看着完全合理的假表

```
SELECT s.name AS student, c.name AS course
FROM students s
JOIN enrollments e ON s.id = e.cid
JOIN courses      c ON e.sid = c.id;
```

实测输出

student	course
韩立	数据库
小明	数据库
韩立	高数
小红	高数
小美	机器学习
小刚	体育

(6 行)

📌 第3课④ 实测反例 2 (🔴 高危：静默错误)

⚠️ 「不为空」绝不等于「对」。唯一的防线是挑一行人工核对内容——这张表里有根本没选课的小美、还有对不上课的小红，一眼就能看出不对

坑③： `ON` 接反 → 0 行（不报错） **本周** **SQL**

`groups.id` 是 1/2/3，`group_members.uid` 是 7/9/7 —— 一个都对不上

```
SELECT persons.name AS person, groups.name AS
grp
FROM groups
JOIN group_members ON groups.id =
group_members.uid
JOIN persons      ON group_members.gid =
persons.id;
```

实测输出

person	grp
--------	-----

(0 行)

📌 第3课④ 实测反例 1 (低危：空的一眼看得出来)

⚠️ `JOIN` 查出空表时的第一反应应该是「怀疑 `ON`」，而不是「数据没了」

证明它到底错在哪：行数一样，内容不同 **本周** **SQL**

同一批数据，正确写法和接反写法都出 6 行；真正差的是其中 2 行的内容

```
SELECT
  (SELECT COUNT(*) FROM (
    SELECT s.name, c.name FROM students s
    JOIN enrollments e ON s.id = e.sid JOIN
    courses c ON e.cid = c.id
  EXCEPT
    SELECT s.name, c.name FROM students s
    JOIN enrollments e ON s.id = e.cid JOIN
    courses c ON e.sid = c.id
  )) AS missing_in_wrong,
  (SELECT COUNT(*) FROM (
    SELECT s.name, c.name FROM students s
    JOIN enrollments e ON s.id = e.cid JOIN
    courses c ON e.sid = c.id
  EXCEPT
    SELECT s.name, c.name FROM students s
    JOIN enrollments e ON s.id = e.sid JOIN
    courses c ON e.cid = c.id
  )) AS rows_that_are_fake;
```

实测输出

missing_in_wrong	rows_that_are_fake
2	2

(1 行)

📌 第3课④ 那张假数据卡的下半句 —— 用 SQL 自己证明

⚠️ 左边 = 正确结果里有、接反结果里没有的（漏了 2 行）；右边 = 接反结果凭空多出来的（假了 2 行）。⇒ 只数行数永远发现不了，必须看内容

诊断三步：先数绳子，再数结果 **本周** SQL

① 单独数绳子表 ② 数 JOIN 结果 ③ 两数一致不算过关，再挑一行核对内容

```
SELECT (SELECT COUNT(*) FROM enrollments) AS rope_rows,
       (SELECT COUNT(*) FROM students s JOIN enrollments e ON s.id = e.sid) AS join_rows;
```

实测输出

```
rope_rows | join_rows
-----+-----
6         | 6
(1 行)
```

📌 第3课⑥ · 本课最实用的迁移技能

⚠️ 数字对不上（空 / 多很多）→ 几乎一定是 ON；数字**对得上**也可能内容错（id 重叠时）

LEFT JOIN **本周** SQL

左表一行都不能丢；右表配不上就整行填 NULL

```
SELECT s.name, e.cid, e.grade
FROM students s LEFT JOIN enrollments e ON s.id = e.sid;
```

实测输出

```
name | cid | grade
-----+-----+-----
韩立 | 1   | 88.5
韩立 | 2   | 76.0
小明 | 1   | 92.0
小明 | 3   | 81.0
小红 | 5   | 59.0
小刚 | 4   | NULL
小美 | NULL | NULL
(7 行)
```

📌 第3课⑤ 换皮3 · 统计时最常见

⚠️ 把 LEFT 去掉，没选课的人会整个消失 —— 人数从 5 变 4，而且不报错

INNER vs LEFT：差在哪一行 **本周** SQL

同一条查询，只差 LEFT 四个字母，结果少一行

```
SELECT s.name FROM students s JOIN enrollments e ON s.id = e.sid;
```

实测输出

```
name
----
韩立
韩立
小明
小明
小红
小刚
(6 行)
```

📌 对照上一张卡看：5 个人变成了 4 个人，小美没了

⚠️ 要「一个都不漏」的清单（点名、对账、覆盖率）时必须 LEFT

看这个库有哪些表 **本周** SQL

不用装数据库工具，一条 SQL 就能看清家底

```
SELECT name FROM sqlite_master WHERE type = 'table' ORDER BY name;
```

实测输出

```
name
-----
courses
enrollments
group_members
groups
persons
students
(6 行)
```

📌 第3课① 「这库里一共几张表」那道题的正确做法

⚠️ sqlite_master 是 SQLite 专有；MySQL 里叫 SHOW TABLES

看一张表的结构 本周 SQL

列名、类型、是否主键，一条 PRAGMA 全给你

```
PRAGMA table_info(enrollments);
```

实测输出

cid	name	type	notnull	dflt_value	pk
0	sid	INTEGER	0	NULL	1
1	cid	INTEGER	0	NULL	1
2	grade	REAL	0	NULL	0

(3 行)

📌 忘了列名时先看这个，别猜

⚠️ PRAGMA 不是标准 SQL（且不吃 ? 占位符）—— 表名只能拼字符串，只用于自己写死的表名

CREATE TABLE 本周 SQL

并列三件事：列定义、主键、外键约束

```
CREATE TABLE enrollments_new (  
    sid INTEGER NOT NULL REFERENCES  
    students(id),  
    cid INTEGER NOT NULL REFERENCES  
    courses(id),  
    grade REAL,  
    PRIMARY KEY (sid, cid)  
);
```

实测输出

cid	name	type	notnull	dflt_value	pk
0	sid	INTEGER	1	NULL	1
1	cid	INTEGER	1	NULL	1
2	grade	REAL	0	NULL	0

(3 行)

📌 第3课③ · D8 作业的第一件事

⚠️ 复合主键 PRIMARY KEY (sid, cid) 直接防住「同一个人重复选同一门课」—— 比在 Python 里查一遍再插可靠

INSERT INTO 本周 SQL

列名写全，值用 ? 占位 (Python 里)

```
INSERT INTO students (id, name, class) VALUES  
(6, '小七', '计科1班');
```

实测输出

id	name	class
6	小七	计科1班

(1 行)

📌 D8 作业：插 3 行数据

⚠️ 列名和值数量必须一致，否则报 table has N columns but M values were supplied

ORDER BY 本月 SQL

排序；默认升序，DESC 反过来

```
SELECT s.name, e.grade  
FROM enrollments e JOIN students s ON s.id =  
e.sid  
ORDER BY e.grade DESC;
```

实测输出

name	grade
小明	92.0
韩立	88.5
小明	81.0
韩立	76.0
小红	59.0
小刚	NULL

(6 行)

📌 出榜单 / 看最大的那个

⚠️ NULL 在 SQLite 里排在最前（DESC 时排最后）—— 想让它垫底要写 ORDER BY grade IS NULL, grade DESC

LIMIT / OFFSET 本月 SQL

只要前 N 条 / 跳过前 N 条 (分页就靠它)

```
SELECT name, credit FROM courses ORDER BY
credit DESC LIMIT 2;
```

实测输出

name	credit
高数	5
数据库	3

(2 行)

📌 Top-N 排行榜; 翻页

⚠️ LIMIT 没有 ORDER BY 时顺序是**未定义**的 —— 每次跑可能不一样

DISTINCT 本月 SQL

一行里多个列都相同才算重复, 去重后只剩一种

```
SELECT DISTINCT class FROM students;
```

实测输出

class
讯飞班
计科1班

(2 行)

📌 「有几种」而不是「有几条」

⚠️ COUNT(DISTINCT 列) 才是「数几种」; COUNT(*) 数的是行数

AND / OR 的优先级 本月 SQL

AND 比 OR 先算 —— 所以 OR 两边一定要加括号

```
SELECT name, class FROM students
WHERE (class = '讯飞班' OR class = '计科1班')
AND name <> '小红';
```

实测输出

name	class
韩立	讯飞班
小明	讯飞班
小刚	计科1班
小美	计科1班

(4 行)

📌 多条件筛选

⚠️ 不加括号会变成「讯飞班的人, 或者 (计科1班且不叫小红的人)」—— 结果多出来的人你还不一定看得出来

IN 本月 SQL

一个字段对一串值, 比写一堆 OR 干净

```
SELECT name FROM students WHERE class IN ('讯飞班', '计科1班');
```

实测输出

name
韩立
小明
小红
小刚
小美

(5 行)

📌 按一批 id 批量查 (爬虫/接口返回的 id 列表)

⚠️ NOT IN 遇到 NULL 会整体变空 —— 子查询里可能有 NULL 时别用

BETWEEN 本月 SQL

区间筛选, 两头都算在内

```
SELECT name, credit FROM courses WHERE credit
BETWEEN 2 AND 5;
```

实测输出

name	credit
数据库	3
高数	5
机器学习	2
英语	2

(4 行)

📌 按时间/分数区间查

⚠️ BETWEEN 2 AND 5 = >= 2 AND <= 5 (含 5) —— 差一个边界是常见 off-by-one

LIKE 通配 本月 SQL

% = 任意多个字符, _ = 正好一个字符

```
SELECT id, name FROM courses WHERE name LIKE '_
数';
```

实测输出

id	name
2	高数

(1 行)

📌 模糊搜索 / 清洗数据时找脏值

⚠️ SQLite 的 LIKE 对 ASCII **大小写不敏感**, 对中文当然无所谓 —— 但换成 PostgreSQL 就敏感了

COUNT(DISTINCT 列) 本月 SQL

数「有几种」，不是「有几条」

```
SELECT COUNT(*) AS rows_all, COUNT(DISTINCT
cid) AS kinds FROM enrollments;
```

实测输出

rows_all	kinds
6	5
(1 行)	

统计去重后的数量（多少门课被选过）

⚠ COUNT(DISTINCT a, b) 在 SQLite 里是**语法错误**，多列去重得用子查询

SUM / AVG / MIN / MAX / ROUND 本月 SQL

五个聚合函数，NULL 一律不参与计算

```
SELECT ROUND(AVG(grade), 1) AS avg_g,
MAX(grade) AS top, MIN(grade) AS low
FROM enrollments WHERE grade IS NOT NULL;
```

实测输出

avg_g	top	low
79.3	92.0	59.0
(1 行)		

算总分/均分；第7章预习要用

⚠ AVG 忽略 NULL，所以分母是「有分数的行数」而不是总行数——拿不准就先 COUNT(*) 和 COUNT(列) 都打出来看

GROUP BY 本月 SQL

把行按某列分成堆，每堆出一行（聚合函数的分母就变成「这一堆」）

```
SELECT cid, COUNT(*) AS n FROM enrollments
GROUP BY cid ORDER BY n DESC;
```

实测输出

cid	n
1	2
5	1
4	1
3	1
2	1
(5 行)	

第7章预习·每个群的成员数这种「分堆统计」

⚠ SELECT 里只能出现**分组列 + 聚合函数**，出现别的列时 SQLite 不报错但会随便挑一行给你（别的数据库直接报错）

HAVING 本月 SQL

筛「堆」。WHERE 筛行、发生在分组前，HAVING 筛堆、发生在分组后

```
SELECT cid, COUNT(*) AS n FROM enrollments
GROUP BY cid HAVING COUNT(*) > 1;
```

实测输出

cid	n
1	2
(1 行)	

第7章预习·「选课人数 > 1 的课」

⚠ 写成 WHERE COUNT(*) > 1 会直接报错——因为 WHERE 执行时还没分组，没有「每堆」这个概念

坑：WHERE 里用聚合函数 本月 SQL

WHERE 比 GROUP BY 先执行，所以它根本看不到 COUNT(*)

```
SELECT cid FROM enrollments WHERE COUNT(*) > 1
GROUP BY cid;
```

实测：真的会这样报错

OperationalError: misuse of aggregate: COUNT()

报错现场

⚠ 看到 misuse of aggregate function 就一个动作：把条件从 WHERE 挪到 HAVING

IFNULL / COALESCE 本月 SQL

NULL 换个默认值再显示（NULL 参与算术会让整行变 NULL）

```
SELECT sid, cid, IFNULL(grade, 0) AS grade FROM
enrollments;
```

实测输出

sid	cid	grade
1	1	88.5
1	2	76.0
2	1	92.0
2	3	81.0
3	5	59.0
4	4	0
(6 行)		

配 LEFT JOIN 用：没匹配上的那侧全是 NULL

⚠ 别用 IFNULL(grade, 0) 去「修」平均值——用 0 冒充「没考」会把均分拉低，该用 WHERE grade IS NOT NULL

坑：`= NULL` 永远查不出东西 本月 SQL

NULL 的意思是「不知道」，所以「不等于不知道」的答案是「我也不知道」

```
SELECT sid, cid FROM enrollments WHERE grade = NULL;
```

实测输出

```
sid | cid
----+----
(0 行)
```

📌 查「没填的那几条」时最常踩

⚠️ 要判 NULL 只能用 `IS NULL` / `IS NOT NULL` —— 这个坑**不**报错、只是静默返回 0 行

UPDATE ... WHERE 本月 SQL

改数据。**没有 WHERE 就是改全表**（没有撤销）

```
UPDATE enrollments SET grade = 90 WHERE sid = 3
AND cid = 5;
```

实测输出

```
sid | cid | grade
----+----+-----
3   | 5   | 90.0
(1 行)
```

📌 补分数、改状态

⚠️ 养成习惯：先把 `WHERE` 那段单拎出来 `SELECT` 一遍，确认命中行数，再改成 UPDATE

CASE WHEN 以后 SQL

SQL 里的 if-else，可以放在 SELECT 里造一个新列

```
SELECT name, CASE WHEN credit >= 3 THEN 'major'
ELSE 'minor' END AS weight
FROM courses;
```

实测输出

```
name      | weight
-----+-----
数据库    | major
高数       | major
机器学习   | minor
体育       | minor
英语       | minor
(5 行)
```

📌 给数据打标签（分数段/等级），打标后直接喂给模型

⚠️ 不写 `ELSE` 时默认是 NULL，不是原值

IS NULL / IS NOT NULL 本月 SQL

判断「这格是不是空的」唯一正确写法

```
SELECT sid, cid FROM enrollments WHERE grade IS NULL;
```

实测输出

```
sid | cid
----+----
4   | 4
(1 行)
```

📌 找脏数据 / 找漏填的记录

⚠️ 反过来记住：`WHERE grade <> 80` **也不会**把 NULL 的行选出来（NULL 不满足任何比较）

DELETE FROM ... WHERE 本月 SQL

删行。同样：**没有 WHERE 就是清空整张表**

```
DELETE FROM students WHERE id = 5;
```

实测输出

```
left_rows
-----
4
(1 行)
```

📌 删脏数据

⚠️ 删之前先 `SELECT COUNT(*)`；真删表用 `DROP TABLE`（连表结构一起没）

CAST 以后 SQL

改类型。SQLite 类型松，但拼接/比较时该转还得转

```
SELECT CAST('42' AS INTEGER) + 1 AS n, CAST(3.7
AS INTEGER) AS i;
```

实测输出

```
n | i
---+--
43 | 3
(1 行)
```

📌 从文本里提出来的数字要参与计算时

⚠️ `CAST(3.7 AS INTEGER)` 是**截断**不是四舍五入（得 3）；要四舍五入用 `ROUND`

UPPER / LOWER / LENGTH 以后 SQL

大小写与长度。注意 LENGTH 数的是**字符**，不是字节

```
SELECT UPPER('abc') AS u, LOWER('ABC') AS l,
LENGTH('数据库') AS n;
```

实测输出

u	l	n
ABC	abc	3
(1 行)		

📌 数据清洗；校验字段长度

⚠️ `LENGTH('数据库')` = 3 (SQLite 数字符)，但
`LENGTH(CAST('数据库' AS BLOB))` = 9 —— 中文是 3 字节

SUBSTR / REPLACE / TRIM 以后 SQL

截取、替换、去首尾空格 —— 清洗三件套

```
SELECT SUBSTR('2026-09-24', 1, 4) AS y,
REPLACE('a-b', '-', '/') AS r, TRIM(' x ') AS t;
```

实测输出

y	r	t
2026	a/b	x
(1 行)		

📌 日志/网页文本清洗

⚠️ SQLite 的 `SUBSTR` 下标从 **1 开始** (Python 从 0 开始)，
`SUBSTR(s, 1, 4)` 才是钱 4 个字符

date('now') / strftime 以后 SQL

SQLite 没有专门的日期类型，日期就是文本，靠这些函数算

```
SELECT date('now') AS today, strftime('%Y-%m',
'now') AS ym,
CAST(julianday('now') - julianday('2026-
09-21') AS INTEGER) AS days;
```

实测输出

today	ym	days
2026-09-23	2026-09	2
(1 行)		

📌 按天统计；算「距今几天」

⚠️ `date('now')` 取的是 **UTC** —— 想按北京时间算要写
`date('now', '+8 hours')`

GROUP_CONCAT 以后 SQL

把一堆行的某列拼成一个字符串（默认用逗号）

```
SELECT cid, GROUP_CONCAT(sid) AS sids FROM
enrollments GROUP BY cid;
```

实测输出

cid	sids
1	1,2
2	1
3	2
4	4
5	3
(5 行)	

📌 「这个群都有谁」 —— 一行拿到全部成员

⚠️ 拼接顺序不保证；要固定顺序得先在外面排序 (SQLite 3.44+
才支持 `GROUP_CONCAT(x ORDER BY y)`)

子查询 以后 SQL

把一条 SELECT 的结果当另一条的条件用

```
SELECT name FROM courses
WHERE id IN (SELECT cid FROM enrollments WHERE
grade > 85);
```

实测输出

name
数据库
(1 行)

📌 两步查询懒得写两遍时

⚠️ `IN` (子查询) 里如果出现 `NULL`，`NOT IN` 会整体返回空
—— 拿不准就用 `EXISTS`

UNION 以后 SQL

上下摞两条结果集；UNION 去重，UNION ALL 不去重（也更快）

```
SELECT name FROM students WHERE class = '讯飞
班'
UNION
SELECT name FROM students WHERE id <= 3;
```

实测输出

name
小明
小红
韩立
(3 行)

📌 合并两张结构相同的表（比如两个月的日志）

⚠️ 两边**列数必须一样**，列名以第一条为准 —— 列对不齐时不报错，只会悄悄错位

六行骨架：连接 → 游标 → 执行 → 取 → 关 **本周**

sqlite3

连 Python 和数据库的全部动作就这六行

```
import sqlite3
conn = sqlite3.connect(":memory:") #
# 文件路径: :memory: = 内存库
cur = conn.cursor()
cur.execute("CREATE TABLE t (x)")
cur.execute("INSERT INTO t VALUES (1), (2)")
conn.commit() #
# 写操作必须 commit
for row in cur.execute("SELECT x FROM t"):
    print(row)
conn.close()
```

实测输出

```
(1,)
(2,)
```

📌 第3课 · 所有 SQL 作业的外壳

⚠️ `commit()` 忘写的后果见下一张卡 —— 它和 `close()` 是两件事

`commit()` vs `close()`: 谁在丢数据 **本周** sqlite3

P-014 的实测：换一个连接去读，看数据还在不在

```
import sqlite3, tempfile, os, shutil
d = tempfile.mkdtemp(); p = os.path.join(d, "k.db")

c = sqlite3.connect(p); c.execute("CREATE TABLE t(x)")
c.execute("INSERT INTO t VALUES (1)")
c.close() #
# 只 close, 不 commit
c2 = sqlite3.connect(p)
print("commit X / close ✓ → 另一个连接读到",
c2.execute("SELECT COUNT(*) FROM t").fetchone()[0], "行")
c2.close()
```

```
c = sqlite3.connect(p); c.execute("INSERT INTO t VALUES (2)")
c.commit(); c.close() #
# 先 commit, 再 close
c2 = sqlite3.connect(p)
print("commit ✓ / close ✓ → 另一个连接读到",
c2.execute("SELECT COUNT(*) FROM t").fetchone()[0], "行")
c2.close(); shutil.rmtree(d, ignore_errors=True)
```

实测输出

```
commit X / close ✓ → 另一个连接读到 0 行
commit ✓ / close ✓ → 另一个连接读到 1 行
```

📌 第3课① 检索题（当场答错的那道）· P-014

⚠️ 丢数据的凶手是 `commit`，不是 `close`。`close()` 忘写的代价是占资源（见下张卡），不是丢数据

close() 忘了写会怎样 本周 sqlite3

连接不关 → 文件被锁住 → 连删都删不掉

```
import sqlite3, tempfile, os, shutil
d = tempfile.mkdtemp(); p = os.path.join(d, "k.db")
c = sqlite3.connect(p); c.execute("CREATE TABLE t(x)"); c.commit()
try:
    os.remove(p) #
    连接还开着, Windows 不让删
    print("删掉了 (没锁)")
except PermissionError as e:
    print("PermissionError:", e)
c.close()
os.remove(p) #
    关掉之后再删就成功了
print("close 之后再删: 成功")
shutil.rmtree(d, ignore_errors=True)
```

实测输出

```
PermissionError: [WinError 32] 另一个程序正在使用此文件, 进程无法访问。:
'E:\\Temp\\tmpd_i50r_1\\k.db'
close 之后再删: 成功
```

📌 P-014 的另一半: close 的真实代价

⚠️ Linux/Mac 上删得掉, 只有 Windows 会当场报 WinError 32 —— 所以这个坑在 Windows 上反而更容易被发现

executemany() 批量插入 本周 sqlite3

一次插 20 行, 别写 20 条 execute

```
import sqlite3
conn = sqlite3.connect(":memory:")
conn.execute("CREATE TABLE s(name TEXT, score REAL)")
rows = [("小明", 88.5), ("小红", 92.0), ("小刚", 76.5)]
conn.executemany("INSERT INTO s VALUES (?, ?)", rows)
conn.commit()
print(conn.execute("SELECT COUNT(*) FROM s").fetchone()[0], "行")
for r in conn.execute("SELECT * FROM s ORDER BY score DESC"):
    print(r)
conn.close()
```

实测输出

```
3 行
('小红', 92.0)
('小明', 88.5)
('小刚', 76.5)
```

📌 灌测试数据 / 批量写爬下来的数据

⚠️ 参数是一个可迭代对象, 每项对应一条语句; 写成 `executemany(sql, (1,2))` 会把它当成两行单列

fetchone / fetchall / 直接遍历 本周 sqlite3

三种取结果的方式, 选中一条就不可能再回头

```
import sqlite3
conn = sqlite3.connect(":memory:")
conn.execute("CREATE TABLE t(x)")
conn.executemany("INSERT INTO t VALUES (?)", [(1,), (2,), (3,)])
cur = conn.cursor()
cur.execute("SELECT x FROM t")
print("fetchone →", cur.fetchone()) #
    拿走第 1 行, 游标前移
print("fetchall →", cur.fetchall()) #
    剩下的全拿走
cur.execute("SELECT x FROM t")
print("遍历 →", [r[0] for r in cur])
conn.close()
```

实测输出

```
fetchone → (1,)
fetchall → [(2,), (3,)]
遍历 → [1, 2, 3]
```

📌 取查询结果

⚠️ 游标是一次性的: `fetchone()` 之后再 `fetchall()` 拿不到已经取走的那行。要重复用就把结果存进 list

`?' 占位符 (防注入) 本周 sqlite3

值永远走参数, 不要用 f-string 拼进 SQL

```
import sqlite3
conn = sqlite3.connect(":memory:")
conn.execute("CREATE TABLE t(name TEXT)");
conn.execute("INSERT INTO t VALUES ('小明')")
who = "小明" OR '1'='1' #
    想注入的人会这么输
n = conn.execute("SELECT COUNT(*) FROM t WHERE name = ?", (who,)).fetchone()[0]
print("参数化 (对) →", n, "行")
bad = "SELECT COUNT(*) FROM t WHERE name = '%s'" % who
n2 = conn.execute(bad).fetchone()[0]
print("拼字符串 (错) →", n2, "行")
conn.close()
```

实测输出

```
参数化 (对) → 0 行
拼字符串 (错) → 1 行
```

📌 所有带用户输入的查询

⚠️ 单元素也必须写成 `(who,)` —— 少了逗号 `(who)` 只是个字符串, 会被当成一个可迭代对象逐字符拆开, 报 Incorrect number of bindings

row_factory: 按列名取值 本周 sqlite3

设一次 `row_factory`，之后 `row['name']` 就能用

```
import sqlite3
conn = sqlite3.connect(":memory:")
conn.row_factory = sqlite3.Row          # ← 就这一行
conn.execute("CREATE TABLE s(id INTEGER, name TEXT)")
conn.execute("INSERT INTO s VALUES (1, '小明')")
row = conn.execute("SELECT id, name FROM s").fetchone()
print("按下标 →", row[0], row[1])
print("按列名 →", row['id'], row['name'])
print("变字典 →", dict(row))
conn.close()
```

实测输出

```
按下标 → 1 小明
按列名 → 1 小明
变字典 → {'id': 1, 'name': '小明'}
```

📌 结果列一多，下标就记不住了

⚠️ `row_factory` 设在 **connection** 上，不是 cursor 上（设 cursor 上是另一套写法）

捕获数据库错误 以后 sqlite3

`IntegrityError` 是约束被违反，`OperationalError` 是表/列名写错

```
import sqlite3
conn = sqlite3.connect(":memory:")
conn.execute("CREATE TABLE s(id INTEGER PRIMARY KEY, name TEXT UNIQUE)")
conn.execute("INSERT INTO s VALUES (1, '小明')")
try:
    conn.execute("INSERT INTO s VALUES (2, '小明')")  # name 重复
except sqlite3.IntegrityError as e:
    print("IntegrityError →", e)
try:
    conn.execute("SELECT * FROM 不存在的表")
except sqlite3.OperationalError as e:
    print("OperationalError →", e)
conn.close()
```

实测输出

```
IntegrityError → UNIQUE constraint failed: s.name
OperationalError → no such table: 不存在的表
```

📌 爬虫/批量写数据时，重复行不该让整个程序崩

⚠️ 两个异常都继承自 `sqlite3.Error`，图省事可以只 `except sqlite3.Error`；但不要裸 `except:` —— 会把键盘中断也吞掉

with: 自动 **commit / rollback** 以后 sqlite3

`with conn:` 出块时自动提交，出异常自动回滚 —— 但不会关连接

```
import sqlite3
conn = sqlite3.connect(":memory:")
with conn:                                #
    成功 → 自动 commit
    conn.execute("CREATE TABLE t(x)")
    conn.execute("INSERT INTO t VALUES (1)")
try:
    with conn:                            #
        失败 → 自动 rollback
        conn.execute("INSERT INTO t VALUES (2)")
        raise ValueError("假装中途出错")
except ValueError as e:
    print("捕获到: ", e)
print("表里剩", conn.execute("SELECT COUNT(*) FROM t").fetchone()[0], "行")
conn.close()
```

实测输出

```
捕获到:  假装中途出错
表里剩 1 行
```

📌 写多步操作时不想每步都 `commit`

⚠️ `with conn:` ≠ `with sqlite3.connect() as conn:`。后者出块时会关连接，很多教程混着讲

json.dumps 本月 标准库

Python 对象 → JSON 字符串 (接口/存盘都用它)

```
import json
d = {"name": "韩立", "score": 88.5, "tags":
["python", "sql"]}
print(json.dumps(d, ensure_ascii=False))
print(json.dumps(d, ensure_ascii=False,
indent=2))
```

实测输出

```
{
  "name": "韩立", "score": 88.5, "tags":
  [
    "python", "sql"
  ]
}
```

🔴 调大模型 API 的请求体; 把结果存成文件

⚠️ `ensure_ascii=False` 不加, 中文会变成 `\u97e9\u7acb`
—— 能用但没法看

json.loads 本月 标准库

JSON 字符串 → Python 对象 (大模型返回的就是个字符串)

```
import json
s = '{"answer": 42, "ok": true, "items": [1, 2]}'
d = json.loads(s)
print(type(d).__name__, d["answer"], d["ok"],
d["items"][1])
print("访问不存在的键 →", end=" ")
try:
    d["nope"]
except KeyError as e:
    print("KeyError", e)
```

实测输出

```
dict 42 True 2
访问不存在的键 → KeyError 'nope'
```

🔴 解析接口/大模型返回

⚠️ `loads` 吃字符串, `load` 吃文件对象 —— 差一个 `s` 是两种用法, 报错信息很像 (都叫 `JSONDecodeError`)

json.dump / json.load 直接读写文件 本月

标准库

省掉 open 那一步

```
import json, tempfile, os, shutil
d = tempfile.mkdtemp(); p = os.path.join(d,
"out.json")
with open(p, "w", encoding="utf-8") as f:
    json.dump({"n": 3, "who": "韩立"}, f,
ensure_ascii=False)
with open(p, encoding="utf-8") as f:
    print("读回来 →", json.load(f))
shutil.rmtree(d, ignore_errors=True)
```

实测输出

```
读回来 → {'n': 3, 'who': '韩立'}
```

🔴 把中间结果存盘, 下次不用重跑

⚠️ 一定要写 `encoding='utf-8'` —— Windows 默认编码不是 `utf-8`, 中文会乱码或直接 `UnicodeEncodeError`

with open(...) 读文件 本月 标准库`with` 会自动关文件, 不用手动 `close`

```
import tempfile, os, shutil
d = tempfile.mkdtemp(); p = os.path.join(d,
"a.txt")
open(p, "w", encoding="utf-8").write("第一行\n第二行\n")
with open(p, encoding="utf-8") as f:
    for i, line in enumerate(f, 1):
        print(i, line.rstrip()) #
rstrip 去掉行尾换行
shutil.rmtree(d, ignore_errors=True)
```

实测输出

```
1 第一行
2 第二行
```

🔴 读日志/读 txt 语料

⚠️ `line` 自带 `\n`, 直接 `print` 会多一个空行 —— 要么 `rstrip()`, 要么 `print` 时加 `end=''`

open(..., 'w') 写文件 本月 标准库

`w` 覆盖、`a` 追加，都要显式写编码

```
import tempfile, os, shutil
d = tempfile.mkdtemp(); p = os.path.join(d, "log.txt")
with open(p, "w", encoding="utf-8") as f:
    f.write("第一次\n")
with open(p, "a", encoding="utf-8") as f:
    # a = 追加，不会覆盖
    f.write("第二次\n")
print(open(p, encoding="utf-8").read().strip())
shutil.rmtree(d, ignore_errors=True)
```

实测输出

第一次
第二次

📌 记运行日志 / 导出结果

⚠️ `'w'` 会立刻清空原文件 —— 想追加却写了 `'w'`，上一次的结果就没了（且没有提示）

os.makedirs(..., exist_ok=True) 本月 标准库

建多层目录；`exist_ok=True` 让「已存在」不报错

```
import os, tempfile, shutil
d = tempfile.mkdtemp()
target = os.path.join(d, "导出", "2026-09")
os.makedirs(target, exist_ok=True)
os.makedirs(target, exist_ok=True) # 第二次不报错
print("建好了 →", os.path.isdir(target))
shutil.rmtree(d, ignore_errors=True)
```

实测输出

建好了 → True

📌 写文件前确保目录存在（爬虫存图、导出报告）

⚠️ 不写 `exist_ok=True`，跑第二遍就 `FileExistsError` —— 这是「只在第二次运行才炸」的经典 bug

os.path.join / exists 本月 标准库

拼路径永远用 join，别手写 `+ '/' +`

```
import os
p = os.path.join("E:\\项目学习", "交互演示", "函数卡.html")
print(p)
print("存在吗 →", os.path.exists(p))
print("目录名 →", os.path.dirname(p))
print("文件名 →", os.path.basename(p))
print("扩展名 →", os.path.splitext(p)[1])
```

实测输出

E:\项目学习\交互演示\函数卡.html
存在吗 → True
目录名 → E:\项目学习\交互演示
文件名 → 函数卡.html
扩展名 → .html

📌 所有涉及文件路径的代码

⚠️ `os.path.join('E:\\a', '/b')` 里第二个参数带开头的斜杠时，前面的路径会被整个丢掉 —— 得到 `/b`

os.listdir / glob 过滤 本月 标准库

列目录，再按后缀筛

```
import os, tempfile, shutil
d = tempfile.mkdtemp()
for n in ["a.txt", "b.txt", "c.csv"]:
    open(os.path.join(d, n), "w").close()
names = os.listdir(d)
print("全部 →", sorted(names))
print("只要 txt →", sorted(n for n in names if n.endswith(".txt")))
shutil.rmtree(d, ignore_errors=True)
```

实测输出

全部 → ['a.txt', 'b.txt', 'c.csv']
只要 txt → ['a.txt', 'b.txt']

📌 批量处理一个文件夹里的文件

⚠️ `listdir` 给的是文件名不是完整路径 —— 要打开它得再 `join` 一次（这是最常见的「文件找不到」原因）

re.findall 本月 标准库

在一段文本里捞出所有符合模式的片段

```
import re
text = "订单 A1001 金额 88 元; 订单 A1002 金额 192 元"
print("所有订单号 →", re.findall(r"A\d{4}", text))
print("所有数字 →", re.findall(r"\d+", text))
print("成对取 →", re.findall(r"(A\d{4}).*?(\d+) 元", text))
```

实测输出

```
所有订单号 → ['A1001', 'A1002']
所有数字 → ['1001', '88', '1002', '192']
成对取 → [('A1001', '88'), ('A1002', '192')]
```

📌 从网页/日志里提取字段

⚠️ 模式字符串前面一定要加 `r` (原始字符串) —— 不加的话 `\d` 会被 Python 先当成转义字符处理

re.search + 分组 本月 标准库

只找第一个; 用括号分组把想要的部分单独取出来

```
import re
m = re.search(r"(\d{4})-(\d{2})-(\d{2})", "今天是 2026-09-24 周四")
if m:
    print("整段 →", m.group(0))
    print("年 →", m.group(1), " | 月 →", m.group(2), " | 日 →", m.group(3))
    print("命名分组写法 →", re.search(r"(?P<y>\d{4})", "2026").group("y"))
    print("没找到时 →", re.search(r"zzz", "abc"))
```

实测输出

```
整段 → 2026-09-24
年 → 2026 | 月 → 09 | 日 → 24
命名分组写法 → 2026
没找到时 → None
```

📌 解析日期、版本号、URL 参数

⚠️ `re.search` 找不到时返回 `None`, 直接 `.group()` 会 `AttributeError` —— 先 `if m:`

re.sub 本月 标准库

按模式替换 (清洗文本的主力)

```
import re
s = "价格: ¥1,299.00 元 (含税)"
print(re.sub(r"[¥,]", "", s))
print(re.sub(r"\s+", " ", s).strip())
print(re.sub(r"(\d+)\.(\d+)", r"\2.\1", "3.14")) # 分组反向引用
```

实测输出

```
价格: 1299.00 元 (含税)
价格: ¥1,299.00 元 (含税)
14.3
```

📌 清洗爬下来的文本 / 归一化输入

⚠️ `re.sub` 默认替换所有匹配; 只想换第一个要加 `count=1`

datetime.now() / strftime 本月 标准库

取当前时间, 格式化成字符串

```
from datetime import datetime
now = datetime.now()
print("原样 →", now)
print("常用 →", now.strftime("%Y-%m-%d %H:%M:%S"))
print("文件名用 →", now.strftime("%Y%m%d-%H%M"))
print("星期日历 →", now.strftime("%Y-%m-%d %A"))
```

实测输出

```
原样 → 2026-09-24 00:31:57.566138
常用 → 2026-09-24 00:31:57
文件名用 → 20260924-0031
星期日历 → 2026-09-24 Thursday
```

📌 给导出文件起名; 记日志时间戳

⚠️ Windows 上 `strftime('%Y-%m-%d')` 没问题, 但文件名里不能有 `:` —— `%H:%M` 直接拿去当文件名会失败

strptime: 字符串 → 时间 本月 标准库

把文本日期变成能计算的对象（转回来才能减）

```
from datetime import datetime
d = datetime.strptime("2026-09-21", "%Y-%m-%d")
print("解析 →", d, type(d).__name__)
print("换一种格式 →", d.strftime("%Y年%m月%d日"))
try:
    datetime.strptime("2026/09/21", "%Y-%m-%d")
except ValueError as e:
    print("格式对不上 →", e)
```

实测输出

```
解析 → 2026-09-21 00:00:00 datetime
换一种格式 → 2026年09月21日
格式对不上 → time data '2026/09/21' does not
match format '%Y-%m-%d'
```

📌 算复习到期日 / 算实习倒计时

⚠️ 格式串必须和输入**严格一致**（2026/09/21 配 %Y-%m-%d 会报 does not match format）

timedelta: 日期加减 本月 标准库

算差几天、N 天后是哪天

```
from datetime import date, timedelta
anchor = date(2026, 9, 23) # 你的倒推起点
for n in (1, 3, 7, 30):
    print(f"第 {n:>2} 天 →", anchor +
          timedelta(days=n))
gap = date(2027, 3, 1) - anchor
print("到 2027.03 还有", gap.days, "天 ≈",
      gap.days // 7, "周")
```

实测输出

```
第 1 天 → 2026-09-24
第 3 天 → 2026-09-26
第 7 天 → 2026-09-30
第 30 天 → 2026-10-23
到 2027.03 还有 159 天 ≈ 22 周
```

📌 1-3-7-30 复习日程；倒推链

⚠️ date - date 得到 timedelta，取天数用 .days；//7 是向下取整的周数（不是四舍五入）

pathlib.Path 以后 标准库

同一件事的现代写法：/ 拼路径、.suffix 取后缀

```
from pathlib import Path
p = Path("E:/项目学习") / "交互演示" / "函数卡.html"
print(p)
print("后缀 →", p.suffix, " | 名字 →", p.stem,
      " | 父目录 →", p.parent.name)
print("存在吗 →", p.exists(), " | 是文件吗 →",
      p.is_file())
```

实测输出

```
E:\项目学习\交互演示\函数卡.html
后缀 → .html | 名字 → 函数卡 | 父目录 → 交互演示
存在吗 → True | 是文件吗 → True
```

📌 新写的脚本（老代码里 os.path 也常见，两种都要能读）

⚠️ Path 对象不能直接丢给老函数当字符串用 —— 必要时 str(p)

collections.Counter 以后 标准库

数词频，一行搞定（比手写字典 +1 干净）

```
from collections import Counter
words = "数据 数据库 数据 模型 数据库 数据".split()
c = Counter(words)
print(c)
print("前三 →", c.most_common(3))
print("'数据' 出现 →", c["数据"], "次")
print("没出现过的词 →", c["不存在"], "次（不报错，返回 0）")
```

实测输出

```
Counter({'数据': 3, '数据库': 2, '模型': 1})
前三 → [('数据', 3), ('数据库', 2), ('模型', 1)]
'数据' 出现 → 3 次
没出现过的词 → 0 次（不报错，返回 0）
```

📌 统计标签分布 / 看数据里哪类最多

⚠️ 取不存在的键返回 0 而不是 KeyError —— 好处是省事，坏处是拼错词也照样得 0，不报错

defaultdict(list): 分组 以后 标准库

按 key 把东西塞进不同的桶, 不用先判断 key 在不在

```
from collections import defaultdict
rows = [("讯飞班", "韩立"), ("计科1班", "小红"),
        ("讯飞班", "小明")]
g = defaultdict(list)
for cls, name in rows:
    g[cls].append(name)
print(dict(g))
print("不存在的班 →", g["不存在"], "(自动建了空 list)")
```

实测输出

```
{'讯飞班': ['韩立', '小明'], '计科1班': ['小红']}
不存在的班 → [] (自动建了空 list)
```

🔴 把一堆 (分类 内容) 聚成 (分类: 内容)

⚠️ 读一个不存在的 key 会顺手创建它 —— 遍历时边读边写会报

dictionary changed size during iteration

random.choice / sample / randint 以后 标准库

抽一个、抽几个不重复的、抽一个整数

```
import random
random.seed(42) #
固定种子 → 结果可复现
print("抽一个 →", random.choice(["A", "B", "C"]))
print("抽2个不重复 →", random.sample(range(1, 50), 2))
print("1~6 的整数 →", random.randint(1, 6))
random.shuffle(lst := [1, 2, 3, 4])
print("打乱 →", lst)
```

实测输出

```
抽一个 → C
抽2个不重复 → [8, 2]
1~6 的整数 → 6
打乱 → [2, 4, 1, 3]
```

🔴 造测试数据 / 切分训练集

⚠️ `random.seed(42)` 是可复现的关键 —— 不设种子, 同样的代码每次都出不同结果, bug 就复现不了

内置 8 张

sorted(key=...) 本月 内置

排序的万能钥匙: key 决定「按什么排」

```
rows = [("小明", 88.5), ("小红", 92.0), ("小刚", 76.5)]
print("按分数 →", sorted(rows, key=lambda r: r[1], reverse=True))
d = {"b": 2, "a": 3, "c": 1}
print("字典按值排 →", sorted(d.items(), key=lambda kv: kv[1]))
print("多级排序 →", sorted(rows, key=lambda r: (-r[1], r[0])))
```

实测输出

```
按分数 → [('小红', 92.0), ('小明', 88.5), ('小刚', 76.5)]
字典按值排 → [('c', 1), ('b', 2), ('a', 3)]
多级排序 → [('小红', 92.0), ('小明', 88.5), ('小刚', 76.5)]
```

🔴 排行榜 / 按时间排日志 / 结果稳定输出

⚠️ `key=` 收到的是函数, 写 `key=r[1]` 会直接报错;

`list.sort()` 是原地改, `sorted()` 返回新列表

int / float / str 类型转换 本月 内置

输入永远是字符串, 要算就得先转

```
print(int("42") + 1, float("3.5") * 2, str(42) + "!")
print("取整 →", int(3.9), "(截断不是四舍五入) | round →", round(3.9))
print("转不了会炸 →", end=" ")
try:
    int("abc")
except ValueError as e:
    print("ValueError:", e)
```

实测输出

```
43 7.0 42!
取整 → 3 (截断不是四舍五入) | round → 4
转不了会炸 → ValueError: invalid literal for int() with base 10: 'abc'
```

🔴 读用户输入、读文件里的数字

⚠️ `int(3.9)` 是 3 (截断); 要四舍五入用 `round()`。

`int('3.5')` 也会报错, 得先 `float()`

f-string 格式化 本月 内置

在字符串里直接写变量，还能控制小数位和对齐

```
name, score = "韩立", 88.4567
print(f"{name} 的分数是 {score}")
print(f"保留两位 → {score:.2f}")
print(f"百分比 → {score / 100:.1%}")
print(f"右对齐 → |{name:>8}| |左对齐 → |{name:
<8}| |补零 → {7:03d}")
```

实测输出

```
韩立 的分数是 88.4567
保留两位 → 88.46
百分比 → 88.5%
右对齐 → |      韩立| |左对齐 → |韩立      | |补
零 → 007
```

📌 拼日志、拼输出的每一处

⚠️ f-string 只在字符串**前面**有 **f** 时生效；`\n` 在 f-string 里是换行，但**表达式里不能有反斜杠** (3.12 前)

len / range 以后 内置

len 数长度；range 造一串数字（Python 里从 **0 开始、不含结尾**）

```
s = "数据库"
print("len →", len(s), "| range(3) →",
list(range(3)), "| range(1,4) →",
list(range(1, 4)))
print("倒着数 →", list(range(3, 0, -1)))
for i in range(len(s)):
    print(i, s[i])
```

实测输出

```
len → 3 | range(3) → [0, 1, 2] | range(1,4) →
[1, 2, 3]
倒着数 → [3, 2, 1]
0 数
1 据
2 库
```

📌 到处都在用

⚠️ `range(1, 5)` 是 1,2,3,4 (**没有 5**) —— 差一个数的 bug 八成来自这里

enumerate 以后 内置

边遍历边拿到序号，别自己维护计数器

```
for i, name in enumerate(["小明", "小红"],
start=1):
    print(i, name)
print("不加 start →", list(enumerate("ab")))
```

实测输出

```
1 小明
2 小红
不加 start → [(0, 'a'), (1, 'b')]
```

📌 要「第几条」的时候

⚠️ `enumerate(x, start=1)` 才是从 1 开始；默认从 **0** 开始

zip 以后 内置

把两个列表按位置配对，一次遍历两个

```
names = ["小明", "小红", "小刚"]
scores = [88, 92, 76]
for n, s in zip(names, scores):
    print(n, s)
print("配对成字典 →", dict(zip(names, scores)))
print("长度不等时 →", list(zip([1, 2, 3],
"ab")))
```

实测输出

```
小明 88
小红 92
小刚 76
配对成字典 → {'小明': 88, '小红': 92, '小刚': 76}
长度不等时 → [(1, 'a'), (2, 'b')]
```

📌 两个来源的数据要一起处理

⚠️ 长度不等时**按短的截断，不报错** —— 数据不齐时会静默丢数据（Python 3.10+ 可用 `strict=True` 让它报错）

sum / min / max 以后 内置

三个聚合内置，都能配 key= 和默认值

```
nums = [3, 1, 4, 1, 5]
print("sum →", sum(nums), " | min →",
      min(nums), " | max →", max(nums))
print("带初值 →", sum(nums, 100))
print("空列表 →", sum([]), end=" / ")
try:
    min([])
except ValueError as e:
    print("min 报错: ", e)
print("按 key 取最大 →", max(["aa", "bbb", "c"],
                              key=len))
```

实测输出

```
sum → 14 | min → 1 | max → 5
带初值 → 114
空列表 → 0 / min 报错: min() iterable argument
is empty
按 key 取最大 → bbb
```

📌 算总分、找最长的那条

⚠️ sum([]) 是 0，但 min([]) / max([]) 会报 ValueError
—— 空列表要先用 default= (3.4+) 或自己判断

any / all 以后 内置

一串条件里「有没有一个真的」 / 「是不是全都真」

```
nums = [2, 4, 6, 8]
print("全是偶数 →", all(n % 2 == 0 for n in
                           nums))
print("有大于 5 的 →", any(n > 5 for n in nums))
print("空列表的默认 →", any([]), all([]))
```

实测输出

```
全是偶数 → True
有大于 5 的 → True
空列表的默认 → False True
```

📌 批量校验（检查所有字段都非空）

⚠️ 空列表时 all([]) 是 True（「全都满足」在逻辑上成立）
—— 拿它当「数据没问题」的判断会漏掉空数据的情况

第三方 2 张

requests.get (只看签名) 以后 第三方

HTTP 请求最常用的库。这条**只实测了签名**，没联网

```
import inspect, requests
print("requests", requests.__version__)
print("get", inspect.signature(requests.get))
print("post 有 params/data/json →",
      all(k in
inspect.signature(requests.post).parameters for
k in ("params", "data", "json")))
```

实测输出

```
requests 2.32.5
get (url, params=None, **kwargs)
post 有 params/data/json → False
```

📌 之后调接口 / 爬虫 / 调大模型 API

⚠️ ⚠️ 这段**没有真的发请求**（不允许联网实测）。用法要点：

`resp.status_code` 判断成败、`resp.json()` 取 JSON、

`resp.text` 取原文、`timeout=` 一定要给（不给会永久挂住）

[外源] 未联网验证

pandas.read_csv (只看签名) 以后 第三方

读表格数据的事实标准。同样**只实测了签名**

```
import inspect, pandas as pd
print("pandas", pd.__version__)
print("read_csv",
inspect.signature(pd.read_csv))
df = pd.DataFrame({"a": [1, 2], "b": ["x",
"y"]})
print(df.to_string(index=False))
```

实测输出

```
pandas 2.3.3
read_csv (filepath_or_buffer: 'FilePath |
ReadCsvBuffer[bytes] | ReadCsvBuffer[str]', *,
sep: 'str | None | lib.NoDefault' =
<no_default>, delimiter: 'str | None |
lib.NoDefault' = None, header: 'int |
Sequence[int] | None | Literal['infer']" =
'infer', names: 'Sequence[Hashable] | None |
lib.NoDefault' = <no_default>, index_col:
'IndexLabel | Literal[False] | None' = None,
usecols: 'UsecolsArgType' = None, dtype:
'DtypeArg | None' = None, engine: 'CSVEngine |
None' = None, converters: 'Mapping[Hashable,
Callable] | None' = None, true_values: 'list |
None' = None, false_values: 'list | None' =
None, skipinitialspace: 'bool' = False,
skiprows: 'list[int] | int |
Callable[[Hashable], bool] | None' = None,
skipfooter: 'int' = 0, nrows: 'int | None' =
None, na_values: 'Hashable | Iterable[Hashable]
| Mapping[Hashable, Iterable[Hashable]] | None'
= None, keep_default_na: 'bool' = True,
na_filter: 'bool' = True, verbose: 'bool |
lib.NoDefault' = <no_default>,
skip_blank_lines: 'bool' = True, parse_dates:
'bool | Sequence[Hashable] | None' = None,
infer_datetime_format: 'bool | lib.NoDefault' =
<no_default>, keep_date_col: 'bool |
lib.NoDefault' = <no_default>, date_parser:
'Callable | lib.NoDefault' = <no_default>,
date_format: 'str | dict[Hashable, str] | None'
= None, dayfirst: 'bool' = False, cache_dates:
'bool' = True, iterator: 'bool' = False,
chunksize: 'int | None' = None, compression:
'CompressionOptions' = 'infer', thousands: 'str
| None' = None, decimal: 'str' = '.',
lineterminator: 'str | None' = None, quotechar:
'str' = '"', quoting: 'int' = 0, doublequote:
'bool' = True, escapechar: 'str | None' = None,
comment: 'str | None' = None, encoding: 'str |
None' = None, encoding_errors: 'str | None' =
'strict', dialect: 'str | csv.Dialect | None' =
None, on_bad_lines: 'str' = 'error',
delim_whitespace: 'bool | lib.NoDefault' =
<no_default>, low_memory: 'bool' = True,
memory_map: 'bool' = False, float_precision:
'Literal[\'high\', \'legacy\'] | None' = None,
storage_options: 'StorageOptions | None' =
None, dtype_backend: 'DtypeBackend |
lib.NoDefault' = <no_default>) -> 'DataFrame |
```

```
TextFileReader'  
a b  
1 x  
2 y
```

📌 之后做数据分析 / 处理表格数据

⚠️ ⚠️ 只验证了 DataFrame 构造，**没有真读 csv 文件**。要点：
`encoding='utf-8'`（中文 csv 常是 `gbk`）、`dtype=` 防手机号
被当数字
[外源] 未读实际文件

函数卡 v1 · 2026.09.24 | 世界：米黄方格纸 · 实验记录本

数据 `_函数卡数据.js`（由 `tools/build_fn_cards.py` 实测生成，别手改） | 主题 `_主题.css` | 重跑：`python`
`tools/build_fn_cards.py`